torch.jit.interface#

https://pytorch.org/docs/stable/generated/torch.jit.interface.html

Description:

Decorate to annotate classes or modules of different types. This decorator can be used to define an interface that can be used to annotate classes or modules of different types. This can be used for to annotate a submodule or attribute class that could have different types that implement the same interface, or which could be swapped at runtime; or to store a list of modules or classes of varying types. It is sometimes used to implement “Callables” - functions or modules that implement an interface but whose implementations differ and which can be swapped out. Example: .. testcode:

Function Signature

```
torch.jit.interface(obj)[source]#
```

Code Examples

```
import torch
from typing import List

@torch.jit.interface
class InterfaceType:
    def run(self, x: torch.Tensor) -> torch.Tensor:
        pass

# implements InterfaceType
@torch.jit.script
class Impl1:
    def run(self, x: torch.Tensor) -> torch.Tensor:
        return x.relu()

class Impl2(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.val = torch.rand(())

    @torch.jit.export
    def run(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.val

def user_fn(impls: List[InterfaceType], idx: int, val:
torch.Tensor) -> torch.Tensor:
    return impls[idx].run(val)

user_fn_jit = torch.jit.script(user_fn)

impls = [Impl1(), torch.jit.script(Impl2())]
val = torch.rand(4, 4)
user_fn_jit(impls, 0, val)
user_fn_jit(impls, 1, val)
```

Detailed Documentation

Documentation

Decorate to annotate classes or modules of different types.

This decorator can be used to define an interface that can be used to annotate classes or modules of different types. This can be used for to annotate a submodule or attribute class that could have different types that implement the same interface, or which could be swapped at runtime; or to store a list of modules or classes of varying types.

It is sometimes used to implement “Callables” - functions or modules that implement an interface but whose implementations differ and which can be swapped out.

Example: .. testcode: